

SECURITE DES RESEAUX INFORMATIQUE POUR LE WEB



Étudiants:

- Samy TALEB
- Ylies VEYRAT

Encadrant:

- Raphael VIERA

Introduction.....	4
I - PortSwigger labs.....	6
II - Analyse de failles sur DVWA : mise en œuvre de tests d'exploitation.....	7
II.1 - File Upload.....	7
II.1.1. Low.....	8
II.1.2 - Medium.....	9
II.1.3 - Hard.....	11
II.2 - File Inclusion.....	12
II.2.1 - Low.....	13
II.2.2 - Medium.....	14
II.2.3 - Hard.....	15
II.3 - Brute Force.....	16
II.3.1 - Low.....	16
II.3.2 - Medium.....	17
II.3.3 - Hard.....	18
II.4 - XSS Reflected.....	19
II.4.1 - Low.....	20
II.4.2 - Medium.....	21
II.4.3 - Hard.....	21
III - Développement contrôlé d'une application volontairement vulnérable.....	23
III.1 - Vulnérabilité 1 : Injection SQL.....	23
III.1.1 - Objectif.....	23
III.1.2 - Mise en place de l'environnement.....	23
Installation de Python.....	23
Création du projet.....	23
III.1.3 - Configuration de l'application.....	24
III.1.4 - Création de la base de données.....	24
III.1.5 - Exploitation de la faille.....	25
➤ Cas 1 : Connexion normale.....	26
➤ Cas 2 : Injection SQL.....	26
III.1.6 - Correction de la vulnérabilité (Contre-mesure).....	26
III.1.7 - Conclusion.....	27
III.2 - Vulnérabilité 2 : Cross-Site Scripting (XSS).....	27
III.2.1 - Objectif.....	27
III.2.2 - Présentation de XSS.....	27

III.2.3 Ajout d'un formulaire de commentaire vulnérable.....	27
III.2.4 - Lancement de l'application.....	28
III.2.5 - Exploitation de la faille.....	28
III.2.6 - Résultat.....	28
III.2.7 - Correction de la vulnérabilité (Contre-mesure).....	29
III.2.8 - Conclusion.....	29
III.3 - Vulnérabilité 3 : Téléversement de fichier non sécurisé.....	30
III.3.1 - Objectif.....	30
III.3.2 - Ajout d'un formulaire de téléversement.....	30
III.3.3 - Upload d'un fichier malveillant.....	31
III.3.4 - Correction de la vulnérabilité (Contre-mesure).....	33
III.3.5 - Conclusion.....	33
Conclusion.....	34

Introduction

À l'ère du tout numérique, les applications web et les objets connectés occupent une place centrale dans notre quotidien. Que ce soit pour consulter son compte bancaire, commander un repas, piloter un système domotique à distance ou encore gérer des infrastructures industrielles, l'interconnexion constante entre les utilisateurs et les systèmes informatiques via le web ne cesse de croître. Cette dépendance croissante à l'égard des services en ligne s'accompagne toutefois d'un risque grandissant : celui des cyberattaques. Chaque jour, des milliers de sites web sont compromis, des données sensibles sont exposées, et des organisations subissent des pertes financières et d'image considérables à cause de vulnérabilités exploitables.

Dans ce contexte, la sécurité des applications web et des infrastructures IoT (Internet of Things) est devenue un enjeu stratégique majeur pour les entreprises, les gouvernements et les utilisateurs. Il ne s'agit plus seulement de développer des fonctionnalités innovantes, mais de garantir un niveau de sécurité élevé dès la conception des systèmes. Or, pour sécuriser efficacement une application, il faut avant tout comprendre comment un attaquant pourrait s'y prendre pour la compromettre.

C'est précisément l'objectif de ce projet : explorer les failles de sécurité les plus courantes dans les applications web, à travers une approche offensive encadrée, connue sous le nom de pentesting (test d'intrusion). En nous mettant dans la peau d'un attaquant, nous allons apprendre à repérer, exploiter et documenter différentes vulnérabilités, tout en proposant des solutions concrètes pour y remédier.

Ce travail s'inscrit dans le cadre du cours de sécurité des réseaux informatiques dispensé à l'École des Mines de Saint-Étienne. Il vise à offrir une immersion pratique dans le domaine du pentesting web, en mettant l'accent sur les aspects techniques, méthodologiques et éthiques liés à la cybersécurité. À travers une série de laboratoires interactifs sur la plateforme PortSwigger, l'analyse de vulnérabilités sur une application existante telle que DVWA, et le développement contrôlé d'une application volontairement vulnérable (ici PyFlaSQL), ce projet nous amène à expérimenter les mécanismes d'attaque et de défense dans des contextes réalistes.

Les objectifs sont multiples :

- Renforcer notre compréhension des OWASP Top 10, les vulnérabilités les plus critiques recensées dans les applications web modernes.

-
- Appliquer des techniques de pentesting pour identifier et exploiter des failles dans des environnements sécurisés.
 - Développer une application vulnérable, en intégrant délibérément certaines failles, afin de mieux comprendre comment elles peuvent naître dès la phase de conception ou de développement.
 - Proposer des contre-mesures efficaces, en nous appuyant sur les bonnes pratiques de développement sécurisé et les recommandations de la communauté OWASP.
 - Prendre conscience des enjeux concrets de la cybersécurité, et de l'importance de l'audit continu dans le cycle de vie logiciel.

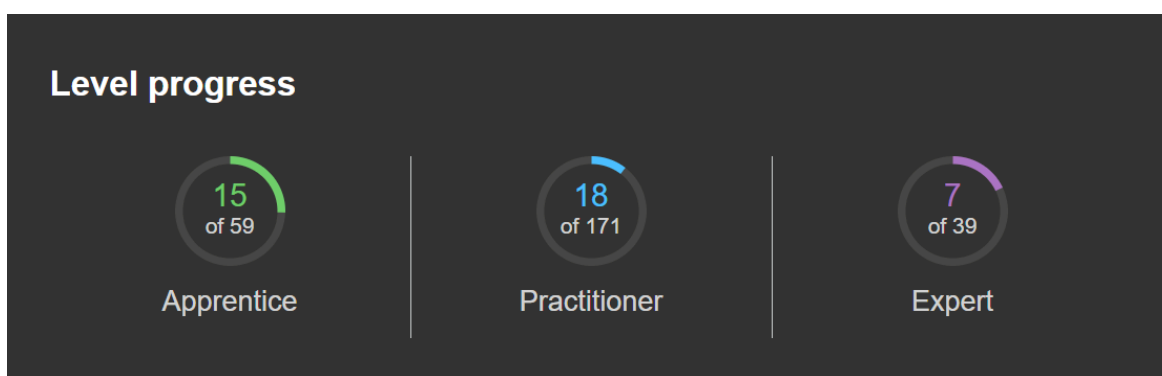
Ce rapport documentera l'ensemble du travail réalisé, en détaillant les méthodes, outils et résultats obtenus. Il sera structuré autour de trois grandes parties : les résultats des laboratoires PortSwigger, les tests d'intrusion menés sur une application vulnérable, et enfin, le développement contrôlé d'une application intégrant plusieurs failles critiques. Cette approche progressive nous permettra non seulement de renforcer nos compétences techniques, mais aussi de développer un regard critique sur la sécurité des systèmes que nous serons amenés à concevoir ou maintenir dans notre future carrière d'ingénieur en informatique.

I - PortSwigger labs

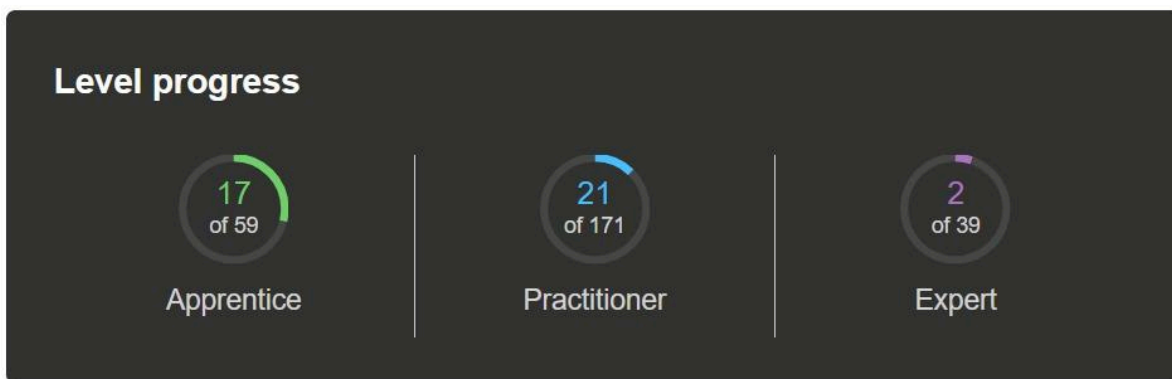
Dans le cadre de ce projet, nous avons chacun réalisé plus de 40 laboratoires interactifs sur la plateforme PortSwigger Web Security Academy. Ces exercices nous ont permis de découvrir et d'exploiter de nombreuses failles de sécurité réelles, allant des injections SQL aux failles XSS, en passant par les erreurs de gestion d'authentification ou de contrôle d'accès.

Vous trouverez ci-dessous nos identifiants ainsi que des captures d'écran attestant de notre progression.

Samy TALEB : samy.taleb@etu.emse.fr KRjLyI7F^Y-K.b:7L2"8-#Ha74\$j;+6H



Ylies VEYRAT : ylies.veyrat@etu.emse.fr 8@4&eA?b}9k3#Z22z\$57F-w8BW8R#)_D



II - Analyse de failles sur DVWA : mise en œuvre de tests d'exploitation

Dans cette partie du rapport, nous mettons en œuvre les connaissances acquises lors des labs PortSwigger en réalisant des tests d'exploitation sur Damn Vulnerable Web Application (DVWA), une application web volontairement vulnérable conçue pour la formation à la sécurité. L'objectif est de confronter la théorie à la pratique en identifiant et en exploitant différentes failles dans un environnement contrôlé.

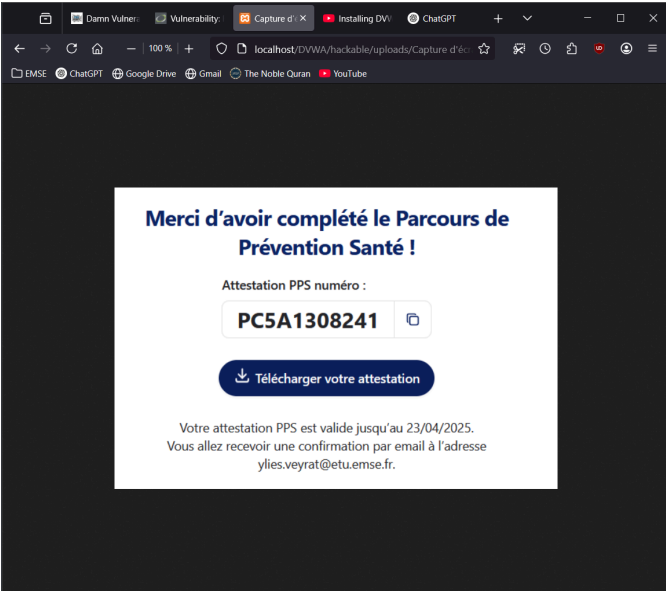
Nous avons ciblé quatre grandes familles de vulnérabilités présentes dans DVWA. Pour chacune, nous avons adapté notre méthode d'attaque aux trois niveaux de sécurité proposés par l'application (low, medium et high), afin d'évaluer la robustesse des protections implémentées. À chaque étape, nous analysons la réussite ou l'échec de l'exploitation et proposons des pistes concrètes de remédiation pour renforcer la sécurité. Ce travail nous permet non seulement de mieux comprendre les vecteurs d'attaque, mais aussi d'adopter une posture défensive face aux menaces web actuelles.

II.1 - File Upload

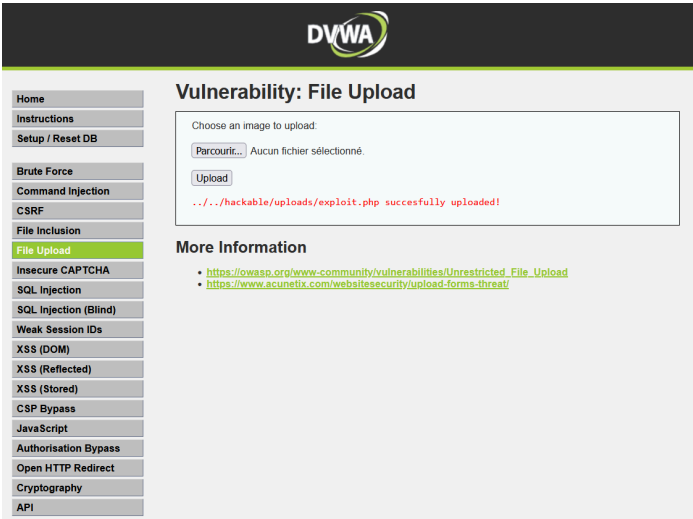
L'objectif de cette partie est d'exploiter les failles liées à la fonctionnalité d'upload de fichiers. Cette fonctionnalité, souvent présente dans les applications web peut représenter un point d'entrée critique si elle est mal sécurisée.

On cherchera ainsi à téléverser un fichier malicieux, comme un script PHP, tout en contournant les éventuels filtres mis en place. Ce challenge permet d'explorer différents niveaux



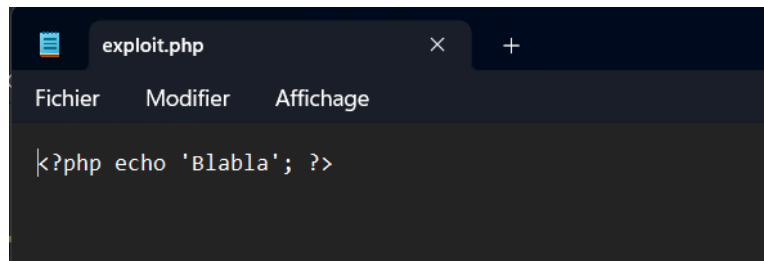


II.1.1. Low



Dans ce scénario, aucun filtre n'est mis en place côté serveur pour contrôler les fichiers uploadés. Cela signifie que l'application accepte tout type de fichier sans vérifier son extension, son contenu, ou son type MIME. C'est une situation critique du point de vue de la sécurité.

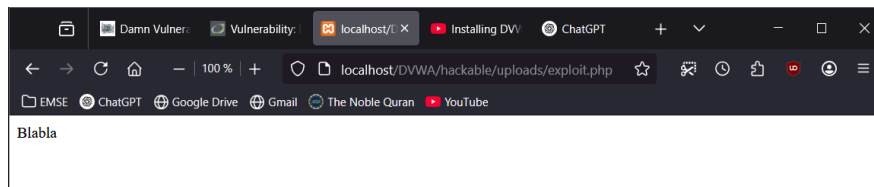
Par exemple, il est possible d'uploader un fichier nommé exploit.php, contenant le code suivant :



```
exploit.php
Fichier  Modifier  Affichage

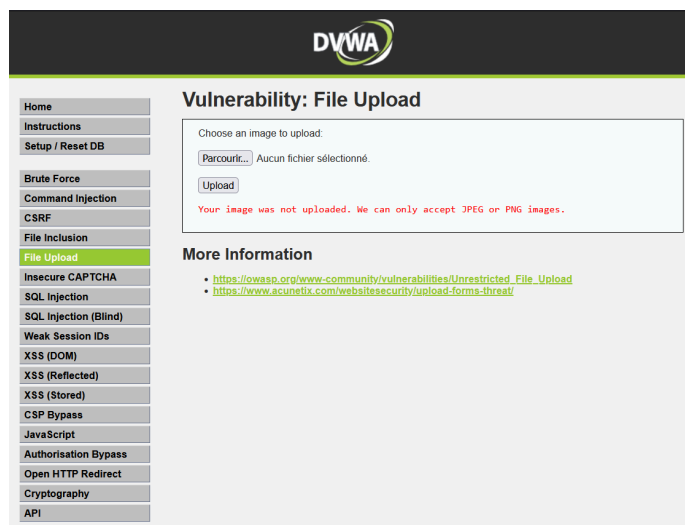
<?php echo 'Blabla'; ?>
```

Une fois ce fichier téléversé avec succès, en accédant à l'URL correspondante , le code PHP est alors interprété par le serveur, et la phrase "blabla" est affichée dans le navigateur. Cela prouve que l'on a pu exécuter du code côté serveur.



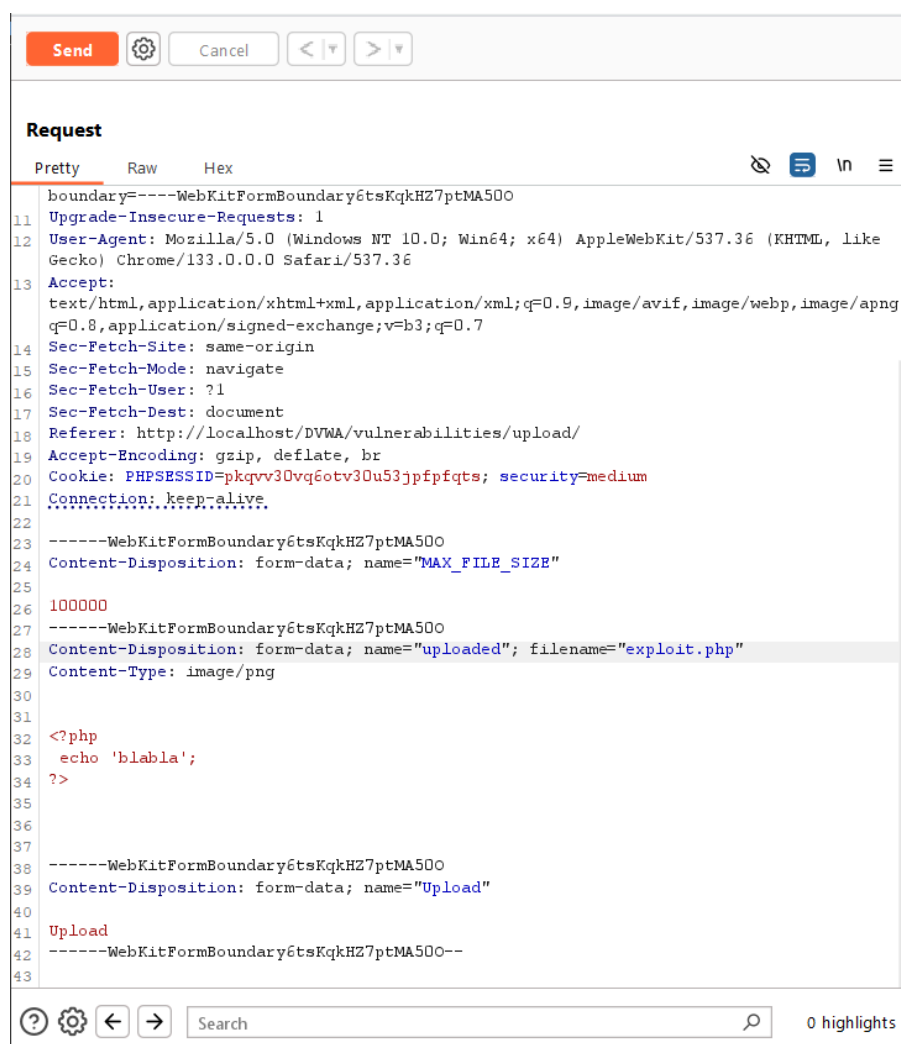
Ce type de faille est extrêmement dangereux, car il ouvre la porte à l'exécution de code arbitraire. En effet, à la place de echo 'blabla';, un attaquant pourrait injecter une commande système, ouvrir une backdoor, lire des fichiers sensibles, ou même prendre le contrôle complet du serveur.

II.1.2 - Medium



À ce niveau, l'application met en place une restriction sur le type de fichier pouvant être téléversé. Elle vérifie le type MIME du fichier envoyé afin de n'accepter que certains formats comme les images (par exemple JPEG ou PNG), dans le but d'éviter l'upload de fichiers potentiellement dangereux comme des scripts PHP.

Cependant, cette vérification repose uniquement sur les informations envoyées par le navigateur dans la requête HTTP, ce qui la rend vulnérable. En effet, on peut facilement modifier ces informations à l'aide d'un proxy comme Burp Suite. En interceptant la requête d'upload, on peut modifier manuellement l'en-tête Content-Type pour faire croire au serveur que le fichier est une image alors qu'il ne l'est pas.



```
boundary=----WebKitFormBoundary6tsKqkHZ7ptMA500
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
13 Gecko) Chrome/133.0.0.0 Safari/537.36
14 Accept:
15 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
16 q=0.8,application/signed-exchange;v=b3;q=0.7
17 Sec-Fetch-Site: same-origin
18 Sec-Fetch-Mode: navigate
19 Sec-Fetch-User: ?1
20 Sec-Fetch-Dest: document
21 Referer: http://localhost/DVWA/vulnerabilities/upload/
22 Accept-Encoding: gzip, deflate, br
23 Cookie: PHPSESSID=pkqv30vq6otv30u53jpfpfqts; security=medium
24 Connection: keep-alive.
25
26 ----WebKitFormBoundary6tsKqkHZ7ptMA500
27 Content-Disposition: form-data; name="MAX_FILE_SIZE"
28
29 100000
30 ----WebKitFormBoundary6tsKqkHZ7ptMA500
31 Content-Disposition: form-data; name="uploaded"; filename="exploit.php"
32 Content-Type: image/png
33
34 <?php
35     echo 'blabla';
36 ?>
37
38 ----WebKitFormBoundary6tsKqkHZ7ptMA500
39 Content-Disposition: form-data; name="Upload"
40
41 Upload
42 ----WebKitFormBoundary6tsKqkHZ7ptMA500--
43
```

II.1.3 - Hard

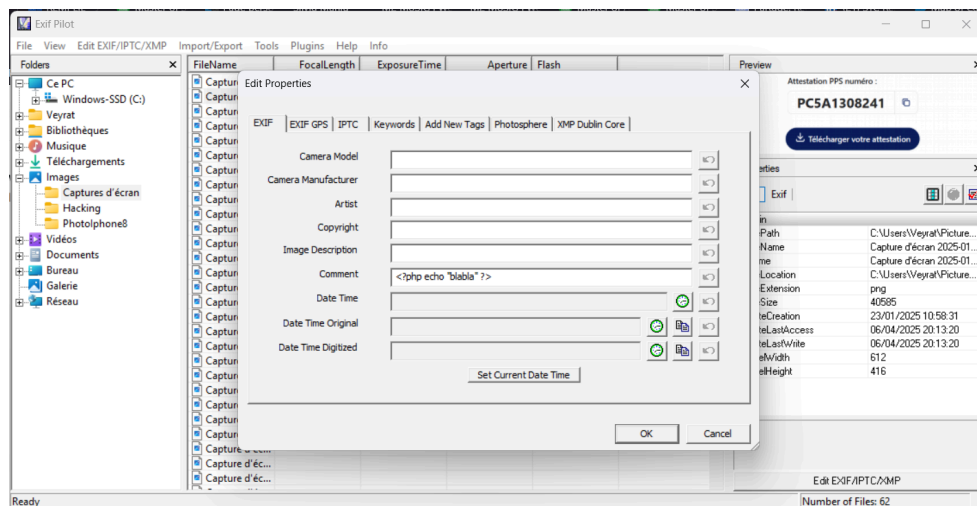
Dans ce niveau de difficulté, les mesures de sécurité mises en place sont nettement plus strictes. Contrairement aux niveaux précédents, la technique consistant à modifier le type MIME avec Burp Suite ne suffit plus. Même en changeant le Content-Type ou l'extension du fichier, l'application refuse systématiquement le téléversement d'un script PHP. Cela signifie que le serveur effectue une vérification approfondie du fichier, probablement en analysant son contenu réel, son extension, et en désactivant l'exécution dans le répertoire d'upload.

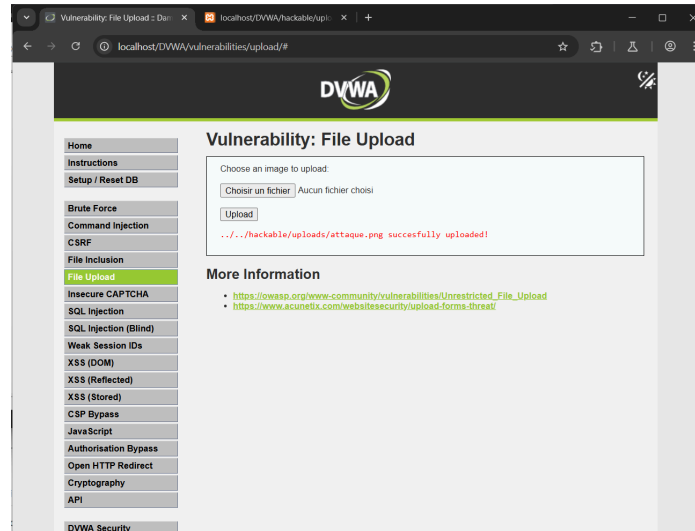
Face à cette restriction, une méthode plus subtile est nécessaire : il s'agit d'injecter la charge malveillante non pas dans le fichier en tant que tel, mais dans ses métadonnées. Pour cela, on utilise un logiciel comme Exif Pilot, qui permet de modifier les données EXIF d'une image

La stratégie consiste à insérer du code PHP directement dans une de ces métadonnées. Une fois cela fait, l'image garde son apparence parfaitement légitime, et le serveur accepte son upload puisqu'il détecte bien une image valide.

Cependant, à ce stade, une question cruciale se pose : comment exécuter le code PHP injecté dans les métadonnées, puisque ce dernier n'est pas dans le corps principal du fichier, mais dans une portion de données qui n'est normalement pas interprétée comme du code ?

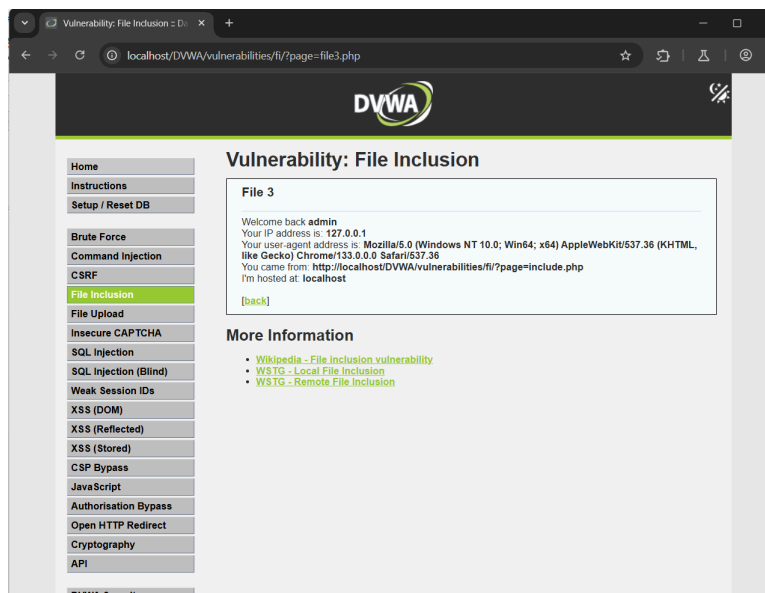
C'est ici que la faille a besoin d'être complétée par une autre vulnérabilité, typiquement une inclusion de fichier. Sans cette faille complémentaire, le code injecté reste stocké passivement et n'est jamais exécuté.





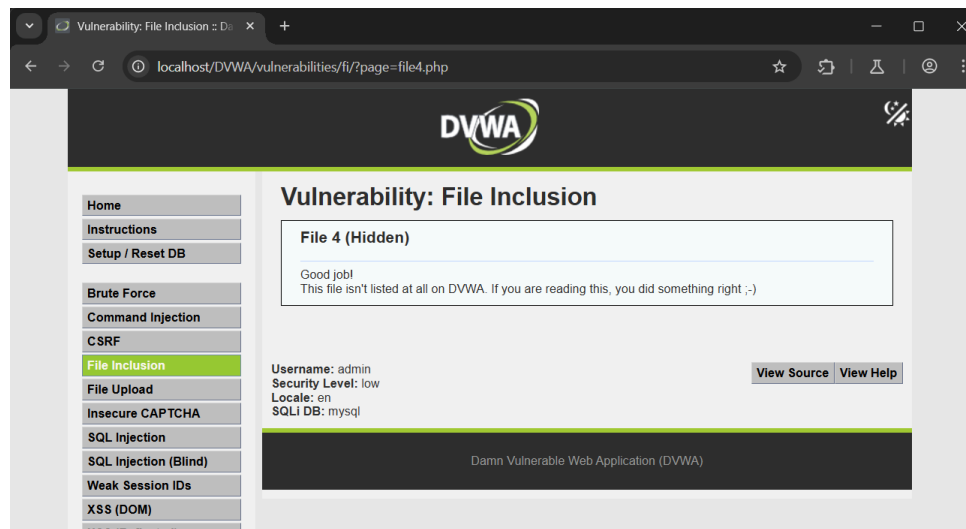
II.2 - File Inclusion

L'objectif de cette partie est d'exploiter une fonctionnalité vulnérable d'inclusion de fichiers, souvent utilisée pour afficher dynamiquement des pages ou charger des ressources. Lorsqu'un paramètre utilisateur est directement utilisé dans une fonction d'inclusion sans validation, cela peut permettre à un attaquant de manipuler le chemin du fichier à inclure. En fonction du contexte, cette faille peut être utilisée pour lire des fichiers sensibles, accéder au code source, ou même exécuter du code malveillant si l'environnement le permet.

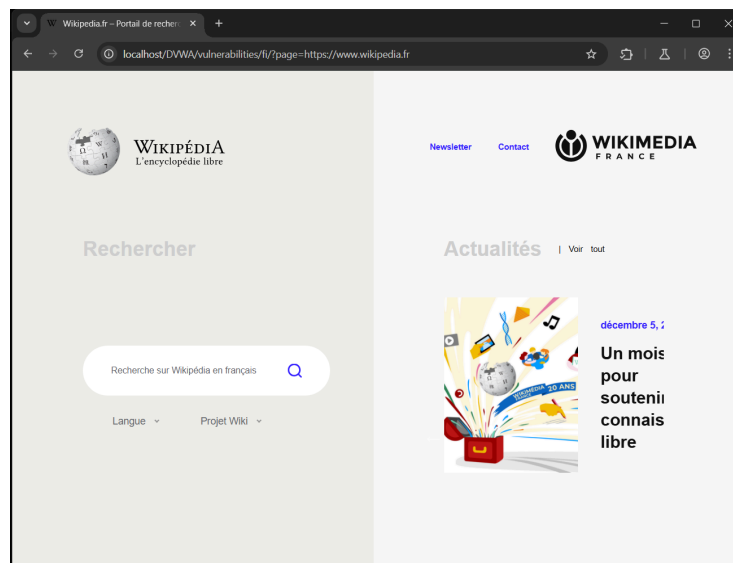


II.2.1 - Low

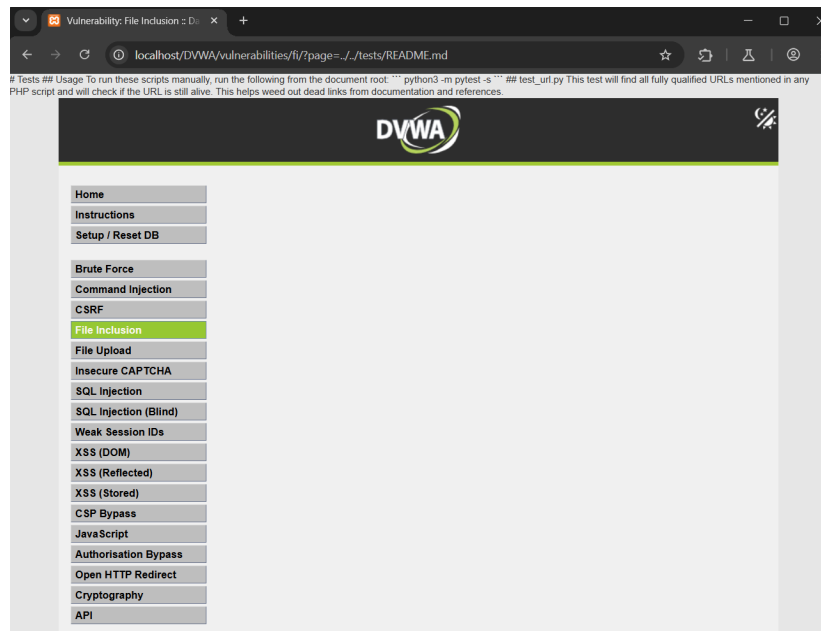
Dans ce premier niveau, l'application propose une fonctionnalité d'inclusion de pages dynamiques via l'URL, par exemple sous la forme ?page=accueil. Trois fichiers sont proposés à l'utilisateur à travers l'interface, et il est possible de naviguer entre eux. Toutefois, en observant l'URL, on comprend rapidement qu'il suffit de modifier la valeur du paramètre page pour tenter de charger d'autres fichiers. On peut accéder à un quatrième fichier qui n'était pas visible dans l'interface, ce qui montre une absence de restriction sur les valeurs acceptées.



Le comportement du site laisse penser que très peu de protections sont mises en place à ce stade, ce qui rend la faille facilement exploitable. En effet, il est possible de charger non seulement des fichiers locaux, mais également des ressources distantes, en utilisant des URLs externes complètes. Ce qui pourrait nous permettre d'inclure des fichiers hébergés ailleurs, contenant du code PHP malveillant.

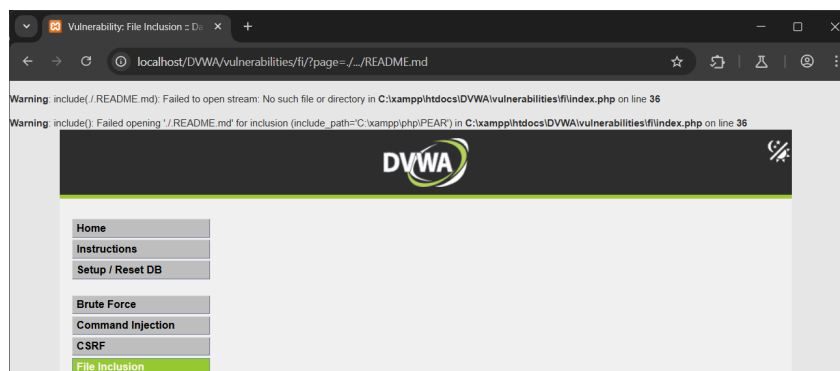


Par ailleurs, on peut aussi exploiter la vulnérabilité avec une attaque de type path traversal, qui consiste à remonter dans l'arborescence du système de fichiers à l'aide de séquences comme `../`. Par exemple, une URL du type `index.php?page=../../../../etc/passwd` peut permettre d'accéder à des fichiers sensibles du système s'il n'y a aucun filtrage sur les chemins. Cela peut aboutir à une fuite d'informations critiques, voire à une exécution de code si l'attaquant parvient à inclure un fichier contenant une charge malveillante.



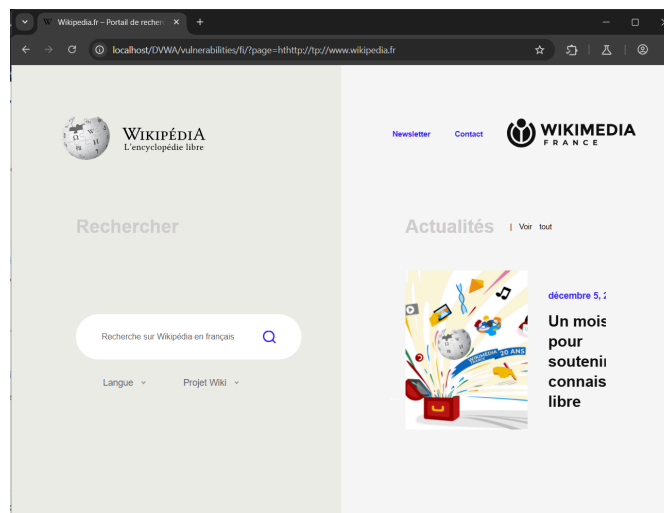
II.2.2 - Medium

À ce niveau, l'application a mis en place des protections plus strictes contre les attaques classiques de type path traversal. Lorsqu'on tente d'accéder à des fichiers locaux sensibles en utilisant des séquences comme `../../../../etc/passwd`, la requête échoue. On essaie alors de contourner ce filtrage avec des variations comme `./.../` ou en jouant sur le double encodage, mais ces tentatives restent infructueuses. Cela suggère que le serveur applique un certain niveau de filtrage ou de normalisation des chemins pour empêcher la navigation vers les répertoires supérieurs.



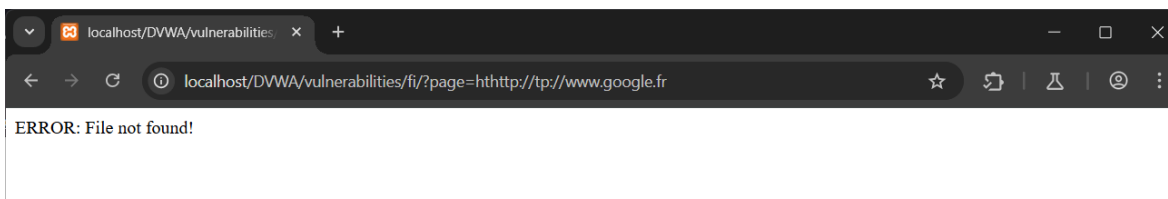
De plus, le serveur semble bien valider et filtrer les chemins pour empêcher l'inclusion de fichiers situés en dehors des répertoires autorisés.

Cependant, l'application reste vulnérable à une autre forme d'exploitation : le contournement des restrictions sur le protocole utilisé pour les URLs. En effet, bien que l'application utilise HTTPS pour sécuriser les échanges, un attaquant peut contourner cette sécurité en manipulant l'URL pour remplacer le protocole `https://` par un protocole invalide comme `hhttp://`. Le serveur, ne validant pas correctement cette partie de l'URL, accepte ces valeurs comme des protocoles valides et tente d'inclure le fichier distant.



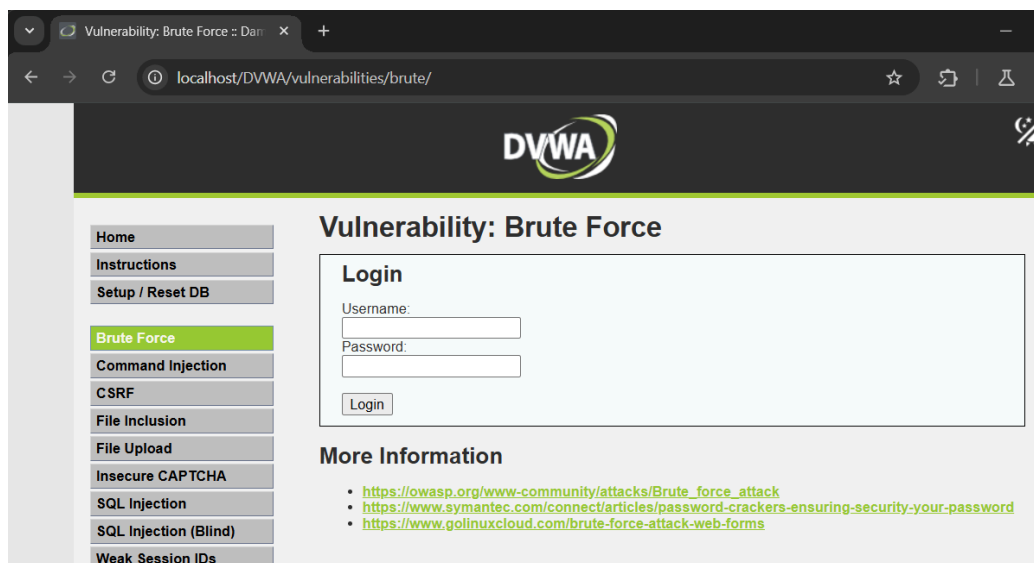
II.2.3 - Hard

Dans ce niveau, l'application semble avoir mis en place des mesures de sécurité robustes pour prévenir les attaques de file inclusion. Les protections contre les attaques de path traversal sont efficaces, et même les tentatives de contournement du protocole via des manipulations simples échouent.



II.3 - Brute Force

L'objectif de cette partie est d'exploiter une vulnérabilité dans le mécanisme d'authentification d'un site en utilisant une attaque par force brute. L'idée est de tenter de deviner le login et le mot de passe d'un utilisateur en essayant toutes les combinaisons possibles ou une liste de mots de passe courants. En fonction des protections mises en place (comme des mécanismes de verrouillage de compte ou des délais entre les tentatives), l'attaque peut être plus ou moins complexe à réaliser.



II.3.1 - Low

Dans ce scénario de niveau faible, l'objectif est de réaliser une attaque par force brute sur un formulaire de connexion avec un login spécifique. En l'occurrence, le login utilisé est admin, qui est souvent un identifiant par défaut dans de nombreuses applications ou systèmes.

L'attaque consiste à tester une série de mots de passe courants et largement disponibles sur Internet. Il existe des listes de mots de passe fréquemment utilisés, comme celles issues de fuites de données précédentes, ou bien des listes générées à partir des mots de passe les plus populaires. Ces listes sont facilement accessibles sur des forums ou des sites spécialisés et contiennent des mots de passe comme 123456, password, admin123, etc.

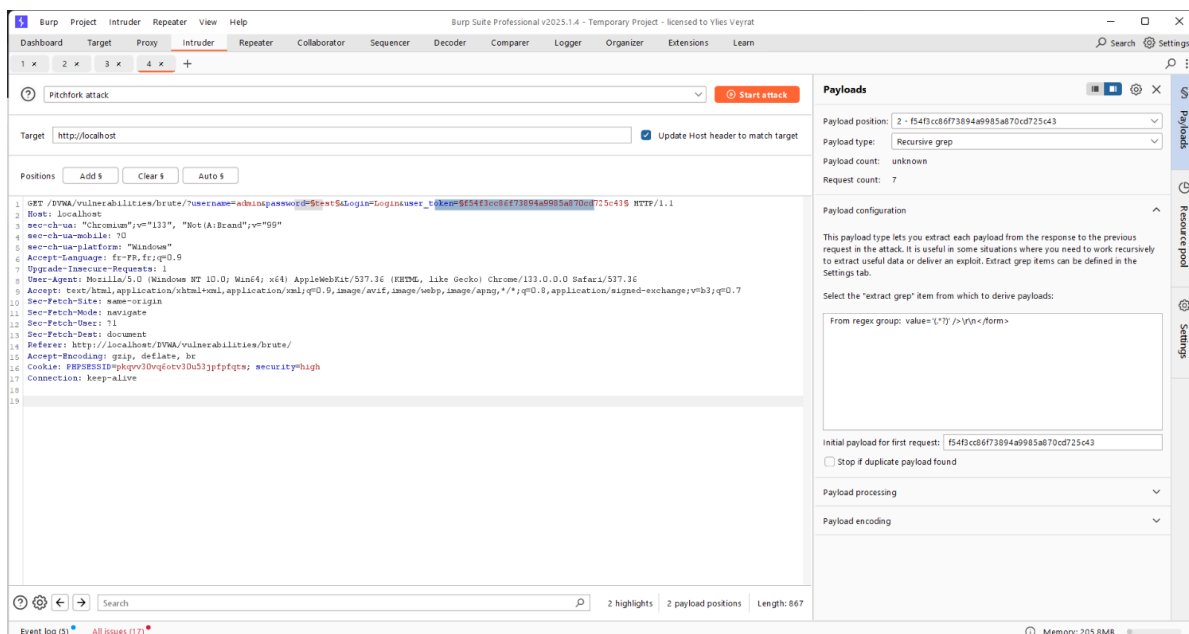
En utilisant Burp Suite, on automatise l'envoi de ces mots de passe au formulaire de connexion.

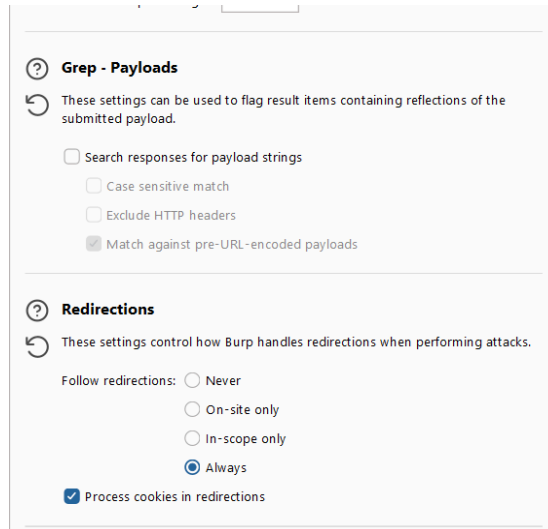
II.3.3 - Hard

À ce niveau, l'application a mis en place des protections avancées pour contrer les attaques par force brute. Tout d'abord, un jeton anti-CSRF de type per-request est ajouté au formulaire de connexion. Ce jeton est unique pour chaque requête, ce qui empêche les attaques automatisées qui essaient de soumettre des formulaires sans prendre en compte la validité du jeton. Le jeton doit être inclus dans la requête pour qu'elle soit valide, et il change à chaque tentative.

De plus, le serveur a également mis en place un mécanisme de délai aléatoire entre chaque tentative de connexion. Plutôt que de limiter les requêtes à une fréquence fixe, le délai entre deux tentatives est désormais variable, compris entre 1 et 3 secondes, rendant l'attaque plus lente et plus complexe à réaliser. Ce délai fluctuant empêche l'envoi rapide de nombreuses requêtes dans un court laps de temps, réduisant ainsi l'efficacité des attaques automatisées.

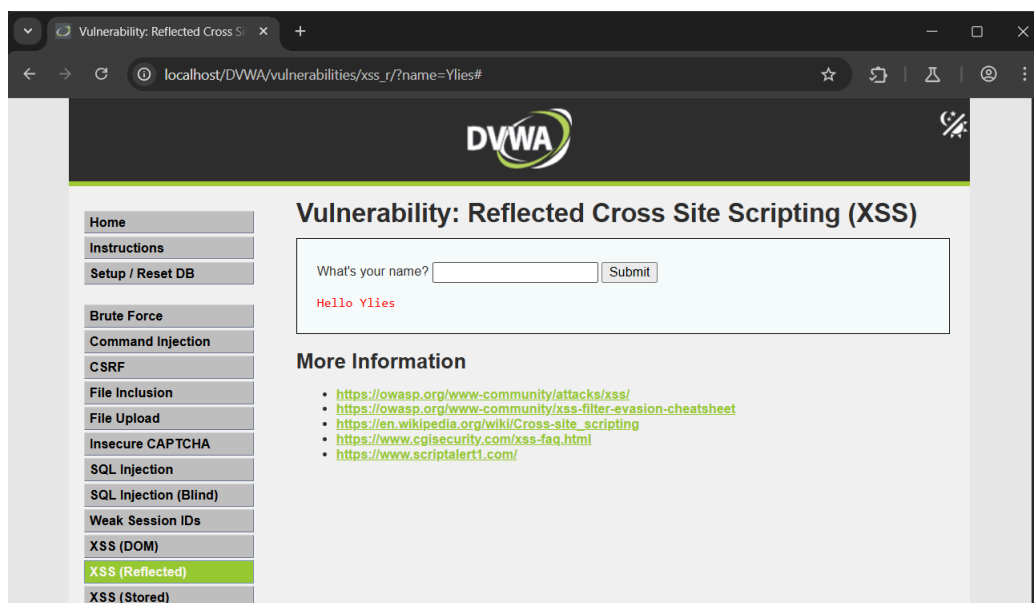
Pour contourner ces protections, on doit d'abord extraire le jeton de la réponse HTTP reçue lors de la soumission d'une première requête. En analysant le contenu de la réponse, le jeton anti-CSRF peut être extrait. Ensuite, on doit réinjecter ce jeton dans la requête suivante afin de garantir que la soumission soit valide. Ce processus doit être répété pour chaque nouvelle tentative de connexion.





II.4 - XSS Reflected

L'objectif de cette partie est d'exploiter une vulnérabilité de type XSS (Cross-Site Scripting) Reflected. Cette faille permet à un attaquant d'injecter du code JavaScript malicieux dans une page web, généralement via un paramètre d'URL. Contrairement à l'XSS stocké, où le script malveillant est enregistré sur le serveur, l'XSS reflété se produit immédiatement lorsque l'utilisateur soumet une requête, et le code est exécuté au moment où la page répond. Cette vulnérabilité permet à l'attaquant de voler des informations sensibles, comme des cookies ou des données de formulaire, ou encore d'exécuter des actions au nom de l'utilisateur.



II.4.1 - Low

Le script typiquement injecté pourrait ressembler à ceci : `<script>alert()</script>`. Ce code malicieux est inséré dans un champ de texte ou un formulaire, et lorsqu'un utilisateur soumet ce formulaire ou accède à la page contenant ce champ, le script est exécuté. Le résultat est une bannière ou une boîte de dialogue d'alerte qui s'affiche dans le navigateur de l'utilisateur, prouvant que le script a été exécuté avec succès.

Cette bannière d'alerte peut être utilisée comme un vecteur pour une attaque plus sérieuse. En effet, un attaquant pourrait modifier ce script afin qu'il redirige l'utilisateur vers une page malveillante qui permet de récupérer un cookie de l'utilisateur.

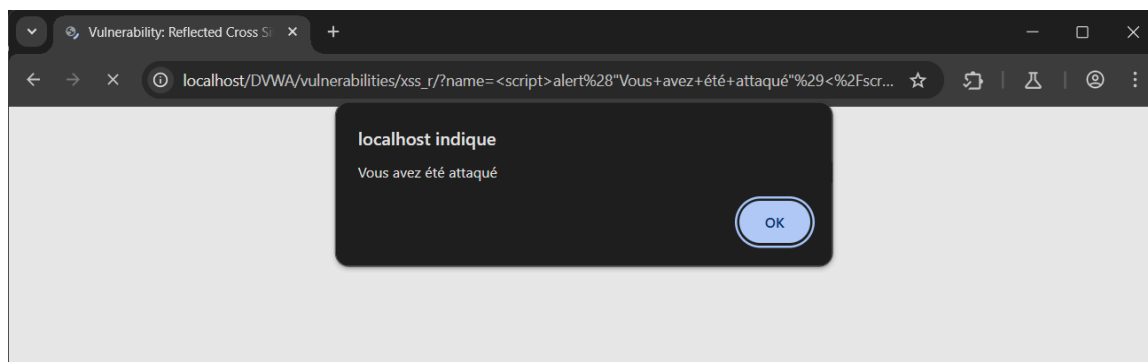
Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More Information

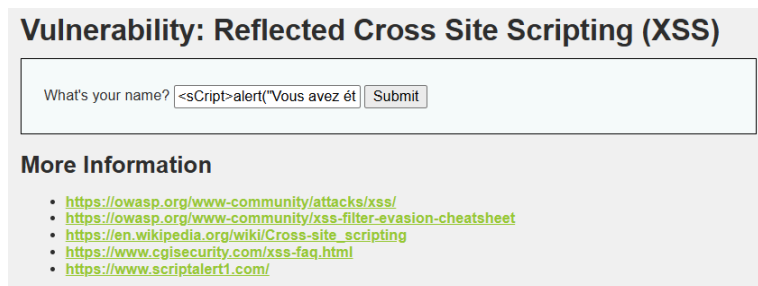
- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>



II.4.2 - Medium

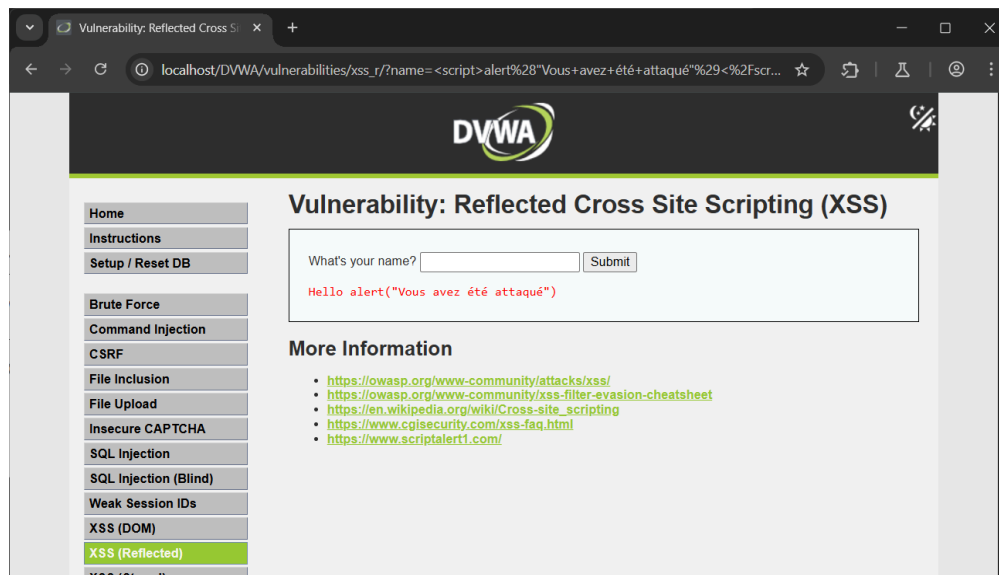
Dans ce niveau, des restrictions ont été mises en place pour empêcher l'injection de scripts malveillants. L'une des principales mesures de sécurité consiste à filtrer certaines balises comme `<script>`. Ainsi, lorsque l'attaquant tente d'injecter un script classique comme `<script>alert()</script>`, le serveur bloque cette tentative, ce qui empêche l'exécution du code JavaScript malveillant.

Cependant, cette restriction peut être contournée en jouant avec la casse des caractères dans le nom de la balise. Par exemple, au lieu d'écrire `<script>`, l'attaquant peut injecter un script avec des variantes en majuscules ou minuscules, comme `<ScRiPt>alert()</ScRiPt>`. Dans ce cas, le filtre du serveur ne détecte pas la balise `<script>` car il n'est pas configuré pour gérer toutes les variations de casse possibles.

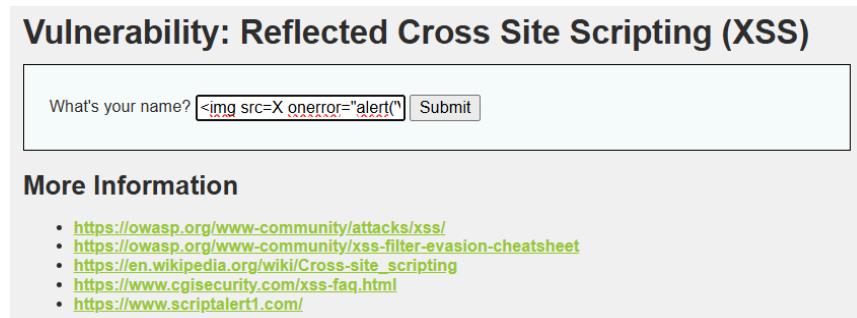


II.4.3 - Hard

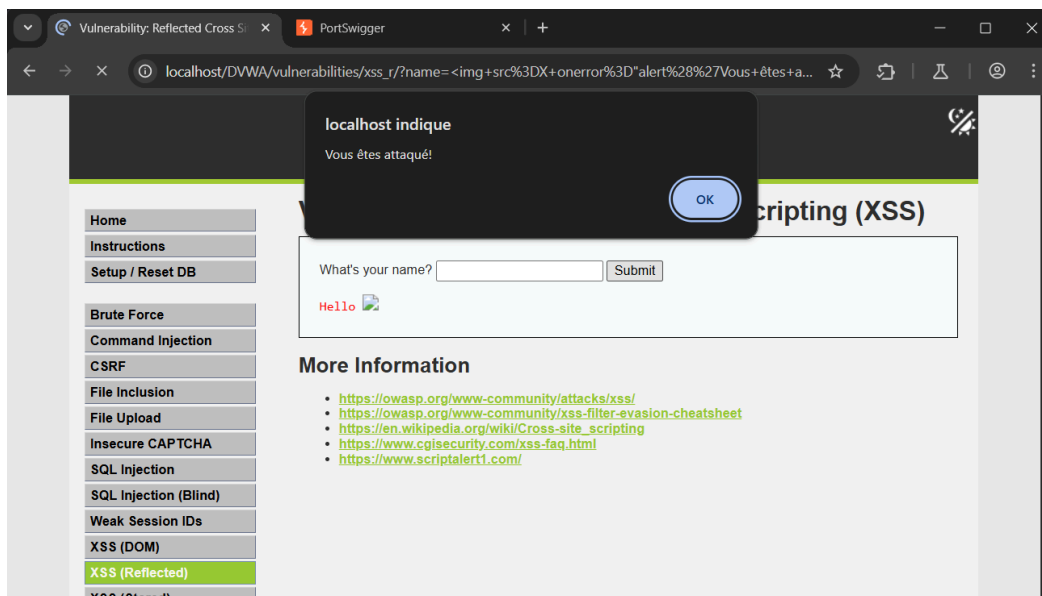
Dans ce niveau, les protections contre les attaques XSS ont été renforcées, ce qui empêche l'attaque précédente avec la bannière de fonctionner correctement. En effet, lorsque l'attaquant tente d'injecter une balise `<script>` ou une version modifiée de celle-ci, le serveur est désormais configuré pour bloquer toutes ces tentatives de manière plus stricte.



Cependant, l'attaquant peut contourner cette restriction en utilisant une autre méthode pour exécuter du code JavaScript malveillant : l'injection via la balise . L'idée est de profiter de l'attribut onerror de l'élément . Lorsqu'une image ne peut pas être chargée (par exemple, si l'URL de l'image est incorrecte), l'événement onerror est déclenché.



Ici, le paramètre src="X" provoque une erreur, car "X" n'est pas une image valide. Cette erreur déclenche alors l'exécution du script JavaScript contenu dans l'attribut onerror.



III - Développement contrôlé d'une application volontairement vulnérable

III.1 - Vulnérabilité 1 : Injection SQL

III.1.1 - Objectif

L'objectif de cette section est de démontrer une vulnérabilité de type Injection SQL en manipulant une requête SQL dans une application web codée avec Flask. Cette faille est volontairement introduite pour illustrer comment une mauvaise gestion des entrées utilisateurs peut compromettre la sécurité d'un système.

III.1.2 - Mise en place de l'environnement

Afin de développer et tester l'application vulnérable, plusieurs outils et dépendances ont été installés sur un environnement Windows.

Installation de Python

Le langage Python est utilisé avec le micro-framework Flask. Python a été installé depuis le site officiel python.org, en s'assurant de cocher l'option "Add Python to PATH" lors de l'installation.

Création du projet

Un dossier `projet-pyflasql` a été créé et l'application PyFlaSQL a été clonée depuis le dépôt Git :

```
git clone https://gitlab.emse.fr/raphael.viera/pyflasql.git
```

```
cd pyflasql
```

Un environnement virtuel Python a ensuite été créé et activé :

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

Les dépendances nécessaires ont été installées :

```
pip install -r requirements.txt
```

III.1.3 - Configuration de l'application

Le fichier principal de l'application s'appelle `run.py` (cf archive projet). L'application a été configurée pour s'exécuter sur le port 4990.

```
from flask import Flask, request, render_template_string
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['GET', 'POST'])
def login():
    html = '''
    <h2>Login</h2>
    <form method="POST">
        Username: <input name="username"><br>
        Password: <input name="password" type="password"><br>
        <input type="submit" value="Login">
    </form>
    <p>{{message}}</p>
    ...
    message = ""
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        # VULNÉRABLE
        conn = sqlite3.connect('users.db')
        cursor = conn.cursor()
        query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
        print(f"[DEBUG] Executing: {query}")
        cursor.execute(query)
        #cursor.execute(f"SELECT * FROM users WHERE username = ? AND password = ?", (username, password)) CORRECTION DE LA VULNERABILITE
        result = cursor.fetchone()
        if result:
            message = "✅ Login successfull"
        else:
            message = "❌ Invalid credentials."
        conn.close()

    return render_template_string(html, message=message)
```

Un nouvel endpoint `/login` a été ajouté, avec un formulaire de connexion vulnérable.

Ce code contient une faille intentionnelle : la requête SQL est construite dynamiquement avec les entrées utilisateur, sans filtre ni protection.

III.1.4 - Création de la base de données

Pour tester l'injection, une base de données SQLite a été créée avec un utilisateur de test. Un script `init_db.py` a été écrit (cf archive projet).


```
import sqlite3

conn = sqlite3.connect('users.db')
cursor = conn.cursor()
cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL,
    password TEXT NOT NULL
)
''')
cursor.execute("INSERT INTO users (username, password) VALUES ('admin', 'admin')")
conn.commit()
conn.close()

print("✅ Base de données créée avec l'utilisateur admin/admin")
```

Cela crée un fichier `users.db` contenant une table `users` avec un utilisateur `admin / admin`.

III.1.5 - Exploitation de la faille

L'application a été démarrée avec :

`python run.py`

```
(venv) C:\Users\samta\projet-pyflasql\pyflasql>python run.py
* Serving Flask app 'run'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:4990
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 106-857-546
127.0.0.1 - - [13/Apr/2025 01:44:58] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [13/Apr/2025 01:45:07] "GET /login HTTP/1.1" 200 -
[DEBUG] Requête exécutée : SELECT * FROM users WHERE username =
'admin' AND password = 'admin'
127.0.0.1 - - [13/Apr/2025 01:45:18] "POST /login HTTP/1.1" 200
-
[DEBUG] Requête exécutée : SELECT * FROM users WHERE username =
'admin' --' AND password = 'uigiugiugig'
127.0.0.1 - - [13/Apr/2025 01:46:28] "POST /login HTTP/1.1" 200
-
```

Elle est accessible à l'adresse :
<http://127.0.0.1:4990/login>

Scénario d'exploitation :

➤ Cas 1 : Connexion normale

- Nom d'utilisateur : **admin**
- Mot de passe : **admin**

➤ Cas 2 : Injection SQL

- Nom d'utilisateur : **admin' --**
- Mot de passe : (n'importe quoi)

Page de Connexion (Vulnérable)

Nom d'utilisateur:

Mot de passe:

✓ Connexion réussie !

Page de Connexion (Vulnérable)

Nom d'utilisateur:

Mot de passe:

✓ Connexion réussie !

La connexion est réussie sans connaître le mot de passe, car la requête SQL devient :

`SELECT * FROM users WHERE username = 'admin' --' AND password = 'xxx'`

Le double tiret (--) annule le reste de la requête (**AND password = ...**), ce qui permet d'accéder au compte sans mot de passe.

III.1.6 - Correction de la vulnérabilité (Contre-mesure)

Pour corriger cette vulnérabilité, la requête SQL a été modifiée pour utiliser des requêtes préparées (paramétrées), ce qui empêche l'interprétation du contenu comme du code SQL.

Version sécurisée :

```
cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?",  
(username, password))
```

Grâce à cette modification, même si un utilisateur saisit **admin' --**, cela sera traité comme une simple chaîne de caractères et la tentative échouera.

Page de Connexion (Vulnérable)

Nom d'utilisateur:

Mot de passe:

✗ Identifiants invalides.

III.1.7 - Conclusion

Cette démonstration illustre clairement comment une simple mauvaise pratique (la concaténation d'entrées utilisateur dans une requête SQL) peut conduire à une faille critique de sécurité. La protection par requêtes préparées est essentielle pour éviter ce type d'attaque.

Cette étape a permis de mieux comprendre la nature et l'exploitation de la vulnérabilité SQL Injection, classée dans le OWASP Top 10 sous la catégorie "Injection".

III.2 - Vulnérabilité 2 : Cross-Site Scripting (XSS)

III.2.1 - Objectif

Cette section illustre l'exploitation d'une vulnérabilité de type Cross-Site Scripting (XSS) dans une application web construite avec Flask. Cette faille permet à un attaquant d'injecter du code JavaScript malveillant dans une page web vue par d'autres utilisateurs.

III.2.2 - Présentation de XSS

Le XSS (Cross-Site Scripting) est une faille très fréquente du OWASP Top 10, permettant l'exécution de scripts injectés par l'utilisateur dans le navigateur d'autres utilisateurs. Elle peut être utilisée pour voler des cookies, rediriger des victimes ou injecter des interfaces de phishing.

III.2.3 Ajout d'un formulaire de commentaire vulnérable

Dans le fichier `run.py`, une nouvelle route `/comment` a été ajoutée (cf archive projet).

```
@app.route('/comment', methods=['GET', 'POST'])
def comment():
    html = '''
        <h2>Laisser un commentaire</h2>
        <form method="POST">
            Votre nom : <input name="username"><br>
            Votre message : <textarea name="message"></textarea><br>
            <input type="submit" value="Envoyer">
        </form>

        <h3>Message reçu :</h3>
        <p><strong>{{username}}</strong> a écrit :</p>
        <div>{{message|safe}}</div> # injection XSS possible
    ...

    username = ''
    message = ''
    if request.method == 'POST':
        username = request.form['username']
        message = request.form['message']

    return render_template_string(html, username=username, message=message)
```

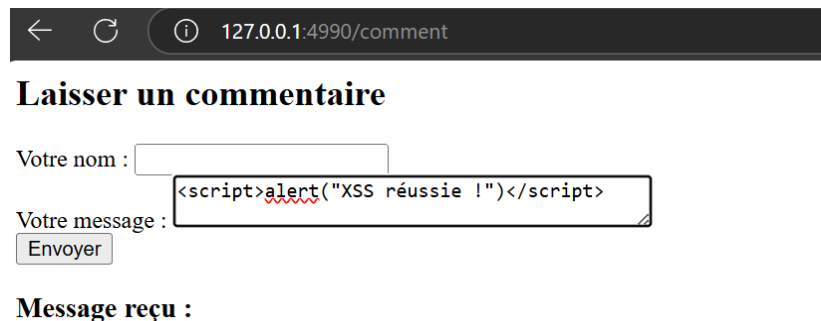
Cette route affiche un formulaire permettant de poster un message. Le contenu du message est ensuite affiché directement sans filtrage, ce qui rend l'application vulnérable au XSS.

III.2.4 - Lancement de l'application

L'application est toujours accessible via :

<http://127.0.0.1:4990/comment>

Un utilisateur peut y saisir un nom et un message. Le message est ensuite affiché dans la page sans être échappé, ce qui crée une faille de type XSS réfléchi (reflected XSS).



← ↻ ⓘ 127.0.0.1:4990/comment

Laisser un commentaire

Votre nom :

Votre message :

Message reçu :

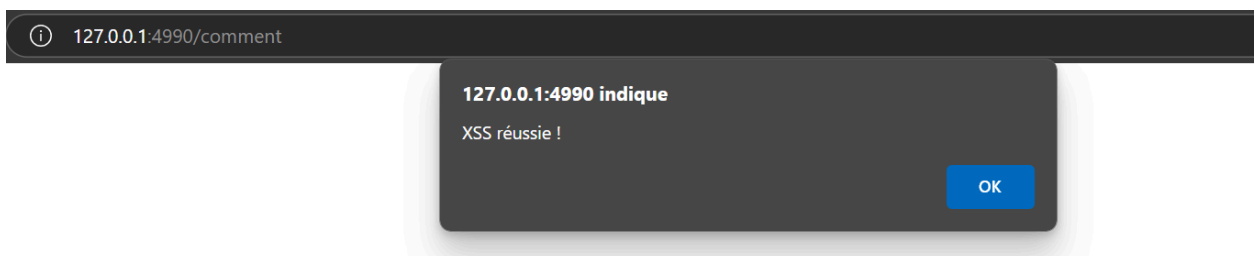
III.2.5 - Exploitation de la faille

Dans le champ "message", un script JavaScript a été injecté :

```
<script>alert("XSS réussie !")</script>
```

III.2.6 - Résultat

Dès validation du formulaire, une alerte JavaScript s'affiche dans le navigateur :



Cela confirme que le script a été exécuté, et donc que le site est vulnérable.

Dans un contexte réel, ce type de script pourrait être utilisé pour :

- Voler des cookies (`document.cookie`)
- Enregistrer des frappes clavier
- Faire du phishing ou détourner l'interface

III.2.7 - Correction de la vulnérabilité (Contre-mesure)

Pour corriger la faille, il suffit d'échapper le contenu injecté afin qu'il soit interprété comme du texte et non du code HTML/JS. Flask utilise Jinja2, qui fait cela automatiquement si on n'utilise pas `|safe`.

Voici la version corrigée :

```
<div>{{ message }}</div> <!-- Pas de |safe => protection automatique →
```

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:4990/comment". The page title is "Laisser un commentaire". Below the title, there is a form with two input fields: "Votre nom :" and "Votre message :". The "Votre message :" field contains the text "# injection XSS possible" followed by a script tag "<script>alert('XSS réussie !')</script>". Below the form, there is a button labeled "Envoyer". The page also displays the text "Message reçu :" and "a écrit :".

III.2.8 - Conclusion

Cette démonstration de faille XSS met en évidence l'importance de ne jamais faire confiance aux données entrantes, même dans les cas simples comme l'affichage d'un commentaire. Il est essentiel de s'assurer que tout contenu affiché dans une page HTML soit échappé pour éviter ce type d'injection. Cette faille est classée dans le OWASP Top 10 sous la catégorie "Cross-Site Scripting", et reste une des plus fréquentes en 2025.

III.3 - Vulnérabilité 3 : Téléversement de fichier non sécurisé

III.3.1 - Objectif

Cette section présente l'implémentation et l'exploitation d'une vulnérabilité de type "Unrestricted File Upload", issue du OWASP Top 10. Cette faille survient lorsqu'un serveur autorise l'envoi de fichiers sans vérifier correctement leur nature ou leur contenu. Cela peut permettre à un attaquant d'exécuter du code malveillant sur le serveur ou de compromettre le système.

III.3.2 - Ajout d'un formulaire de téléversement

Un nouveau endpoint `/upload` a été ajouté dans le fichier `run.py`. Ce formulaire permet à un utilisateur de téléverser une image `.jpg`. Cependant, dans un premier temps, aucun contrôle strict n'est appliqué, ce qui ouvre la voie à une attaque.

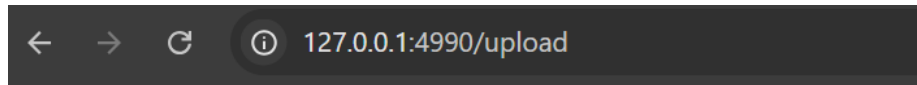
```
import os
from werkzeug.utils import secure_filename

UPLOAD_FOLDER = 'uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

@app.route('/upload', methods=['GET', 'POST'])
def upload():
    html = '''
        <h2>Upload d'image (Vulnérable)</h2>
        <form method="POST" enctype="multipart/form-data">
            Choisissez un fichier (.jpg) : <input type="file" name="file"><br>
            <input type="submit" value="Uploader">
        </form>
        <p>{{ message }}</p>
    ...
    message = ''
    if request.method == 'POST':
        file = request.files['file']
        if file:
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            message = f'✅ Fichier {filename} uploadé avec succès.'

    return render_template_string(html, message=message)
```

Ce code crée un formulaire d'upload et enregistre les fichiers reçus dans le dossier `uploads`, sans aucune vérification de l'extension ni du contenu réel.



Upload d'image (Vulnérable)

Choisissez un fichier (.jpg) : Aucun fichier choisi

III.3.3 - Upload d'un fichier malveillant

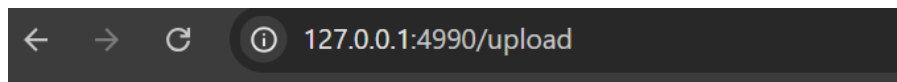
L'utilisateur est censé envoyer des fichiers `.jpg`, mais il est possible d'envoyer un fichier `.php`, `.py` ou tout autre format exécutable si le serveur le permet.

Exemple d'attaque:

Un fichier nommé `test.php` a été créé avec ce contenu :

```
<?php echo "Code PHP exécuté !"; ?>
```

Ce fichier a ensuite été renommé en `test.jpg` pour tromper l'interface.



Upload d'image (Vulnérable)

Choisissez un fichier (.jpg) : Aucun fichier choisi

✓ Fichier test.jpg uploadé avec succès.

Avec Burp Suite, la requête POST a été interceptée, et le nom du fichier a été remis en `.php` avant d'être envoyé :

Target Dashboard Proxy Intruder Repeater Collaborator Sequencer Comparer Logger Organizer Extensions Learn

1 x +

Send Cancel < >

Request

Pretty Raw Hex

```

1 POST /upload HTTP/1.1
2 Host: 127.0.0.1:4990
3 Content-Length: 212
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not:A-Brand";v="24", "Chromium";v="134"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: fr-FR,fr;q=0.9
9 Origin: http://127.0.0.1:4990
10 Content-Type: multipart/form-data;
11 boundary=----WebKitFormBoundaryvWTU4R8ajfsdBx1B
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
14 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
15 Accept:
16 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
17 ebp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
18 Sec-Fetch-Site: same-origin
19 Sec-Fetch-Mode: navigate
20 Sec-Fetch-User: ?1
21 Sec-Fetch-Dest: document
22 Referer: http://127.0.0.1:4990/upload
23 Accept-Encoding: gzip, deflate, br
24 Connection: keep-alive
25
26 -----WebKitFormBoundaryvWTU4R8ajfsdBx1B
27 Content-Disposition: form-data; name="file"; filename="test.php"
28 Content-Type: php
29
30 <?php echo "Code PHP exécuté !"; ?>
31 -----WebKitFormBoundaryvWTU4R8ajfsdBx1B--

```

Response

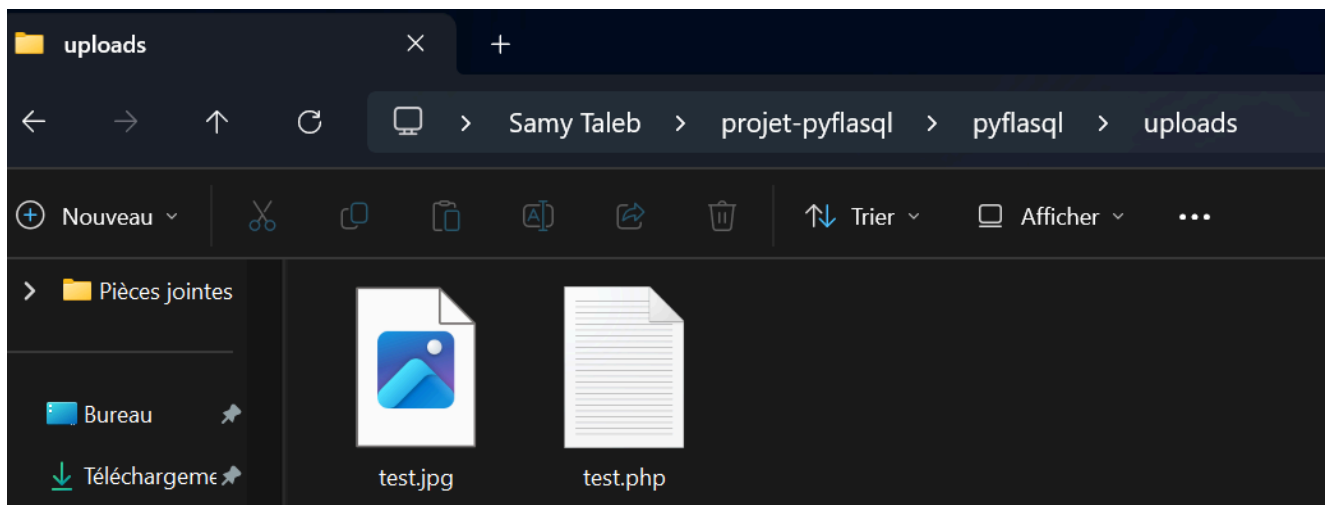
Pretty Raw Hex Render

```

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.3 Python/3.13.0rc3
3 Date: Sun, 13 Apr 2025 02:34:42 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 315
6 Connection: close
7
8
9 <h2>
10 Upload d'image (Vulnérable)
11 </h2>
12 <form method="POST" enctype="multipart/form-data">
13 Choisissez un fichier (.jpg) : <input type="file" name="file">
14 <br>
15 <input type="submit" value="Uploader">
16 </form>
17 <p>
18 □ Fichier test.php uploadé avec succès.
19 </p>
20

```

Le fichier est bien sauvegardé dans le dossier uploads/, et peut être accédé via : <http://127.0.0.1:4990/uploads/test.php> . Sur un serveur de production avec support PHP, cela permettrait une exécution de code distant (RCE).



III.3.4 - Correction de la vulnérabilité (Contre-mesure)

Pour corriger cette vulnérabilité, plusieurs bonnes pratiques peuvent être mises en œuvre comme par exemple la création d'une fonction de vérification de l'extension. En voici un exemple de code:

```
ALLOWED_EXTENSIONS = {'jpg', 'jpeg', 'png'}

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# Mise à jour du bloc POST
if file and allowed_file(file.filename):
    filename = secure_filename(file.filename)
    file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
    message = f'✅ Fichier {filename} uploadé avec succès.'
else:
    message = '❌ Extension non autorisée.'
```

III.3.5 - Conclusion

Cette démonstration met en évidence comment une simple fonctionnalité d'upload, mal protégée, peut introduire une faille critique. Elle rappelle l'importance de valider et filtrer systématiquement les fichiers reçus depuis le client.

La vulnérabilité d'upload non sécurisé figure dans le OWASP Top 10 sous la catégorie "Security Misconfiguration" et "Insecure Design".

Conclusion

Ce projet nous a permis de nous plonger concrètement dans le domaine de la sécurité des applications web et IoT à travers une approche offensive et pédagogique. En réalisant des tests d'intrusion sur des environnements volontairement vulnérables (DVWA et PyFlaSQL), nous avons pu expérimenter les techniques utilisées par les attaquants pour compromettre un système, et mesurer l'impact réel des failles de sécurité les plus critiques recensées dans le classement OWASP Top 10.

L'ensemble du travail effectué, de la résolution des labs sur la plateforme PortSwigger jusqu'au développement contrôlé d'une application vulnérable, nous a offert une vision globale des enjeux liés à la cybersécurité moderne. Ces exercices pratiques nous ont permis de comprendre en profondeur les mécanismes d'exploitation des vulnérabilités telles que :

- l'injection SQL, qui permet de manipuler ou contourner les requêtes vers la base de données ;
- les attaques XSS, qui exploitent le rendu non sécurisé de contenu utilisateur dans les pages HTML ;
- l'upload de fichiers non sécurisé, qui permet à un attaquant de faire exécuter du code malveillant sur le serveur.

Au-delà de l'identification et de l'exploitation de ces failles, un effort particulier a été consacré à la recherche de contre-mesures efficaces, et à la mise en œuvre de bonnes pratiques de développement sécurisé (validation des entrées, échappement des données, vérification des extensions, usage de requêtes préparées...).

Enfin, ce projet a renforcé notre conscience des responsabilités liées à la profession d'ingénieur informatique. La sécurité ne doit pas être perçue comme une couche additionnelle, mais bien comme un principe fondamental à intégrer dès la conception d'un système. Le fait de passer par la phase offensive (attaque) nous a permis de mieux concevoir les défenses à adopter, et d'appréhender les risques d'un point de vue réaliste.

Nous terminons ce projet avec une meilleure maîtrise des outils de pentesting (Burp Suite, SQLMap, etc.), des environnements d'évaluation (DVWA, Mutillidae, PyFlaSQL) et surtout, une solide base de réflexion sur la sécurisation des systèmes que nous serons amenés à concevoir ou auditer dans notre future carrière.